



Application of curiosity driven exploration methods for hardware interference identification

DATE 23/02/2026
REFERENCE REF
ISSUE V1

MATAR Ludovic	Centre Inria de l'Université de Bordeaux, Flowers	Research And Development Engineer
----------------------	--	--------------------------------------

Contents

1	Introduction	3
2	Application of IMGEP for hardware interference identification	3
2.1	Motivation	5
2.2	Related works	6
2.3	Description of our dual-core model	6
2.3.1	Core model	6
2.3.2	Cache model	7
2.3.3	DDR model	8
2.3.4	DDR controller model	8
2.4	Overview of the exploration strategy	9
2.5	Parameter space	10
2.6	Behavior space	11
2.7	Goal sampling	12
2.8	Goal achievement strategy	13
3	Results	14
3.1	Diversity evaluation	14
3.1.1	Method	14
3.1.2	Diversity analysis	15
3.2	Analysis	17
4	Conclusion	18
	References	18
	Appendices	19
A	Description of our dual-core model	19
A.1	Complementary information on the DDR	19
A.2	Complementary information on the DDR controller	19
A.3	Interconnect model	19
A.4	Configuration	20
B	Intrinsically Motivated Goal Exploration Process	20
B.1	Description of IMGEP	20
B.2	Mixing operator	21
B.3	Mutation operator	22
B.4	Selection of parameters	23
C	Evaluation	23
C.1	Diversity	23

1 Introduction

A major contribution of the Flowers team in recent years has been the development of curiosity-driven learning algorithms in order to efficiently explore the diversity of behaviors of complex systems [6]. A complex system is a system consisting of multiple entities interacting with each other, giving rise to emergent phenomena that are difficult to predict. Examples of complex systems in Nature are water molecules crystallizing into snow flakes, collective animal behavior forming complex patterns (e.g. bird flock), or chains of amino acids forming complex 3D protein structures. Complex system dynamics can also be simulated in silico, e.g. in cellular automata such as the Game of Life. Such dynamics are even present at the hardware level, e.g. in multi-core architectures exhibiting complex interference patterns due to the concurrent execution of multiple programs on shared computing resources. Understanding, simulating and being able to predict the behavior of complex systems is an important challenge in many scientific fields. For this aim, scientists often need to build a model mapping the input parameters of the system (e.g. a program of amino acids) to its corresponding observed behavior (e.g. the resulting 3D shape of a protein). This is a challenging problem because the parameter space is often in high dimensions and the mapping from parameters to behaviors is highly non-linear and unpredictable. To solve this problem, the Flowers team has proposed Automated Discovery algorithms. The objective of these algorithms is to automatically discover all the diversity of behaviors a complex system can exhibit, with a limited experimental budget. Such algorithms were originally developed as a model of how children or artificial agents proactively explore their environment ([2], 2013, hence the term “curiosity-driven”) and we have recently developed a framework for using them to explore any type of complex systems ([6], chapters 1,2,3, 2023).

In this work, we aim to show that Automated Discovery (AD) algorithms are suited to provide a significant help for discovery tasks requiring multiple interactions with SOC (System on chips), that is, integrated circuits combining key most components on a computer. Under some practical considerations, SOC can be considered as complex systems. Therefore, as an example to demonstrate the utility of AD algorithms, we propose an application on a very concrete use case: the discovery of diverse interference patterns in multi-core hardware architectures. In particular, we work with the algorithm Intrinsically Motivated Goal Exploration Process (see [6], chapter 1) that we apply on a simplified simulated dual-core model.

In the following, we unroll our work according to the plan below, we:

1. propose the study case of the discovery of diverse interference patterns in multi-core hardware architectures.
2. present Intrinsically Motivated Goal Exploration Process algorithm architecture and apply it on our dual-core architecture simulator.
3. We present a summary of our findings, with results of diversity measures.

2 Application of IMGEP for hardware interference identification

The shift from single-core to multi-core architectures is essential in safety-critical embedded systems in multiple domains such as aerospace and automotive, driven by both the need to enhance processor performance for increasingly demanding applications and adaptation to recent technology. However, this transition introduces new complexities, particularly hardware contention issues known as inter-core interferences. As the occurrence of interference has a consequence on the execution time, the use of multi-core processors in critical real-time systems represents a challenge. Indeed, temporal requirements with high levels of confidence have to be met. In the avionics domain, the AMC 20-193 standard requires to identify all sources of interference that can occur when executing tasks in parallel and to address them.

Due to the hardware complexities, the conditions under which interference occurs, as well as their effects, can vary greatly and often seem random, making them very difficult to model and to predict. In other words, such architectures are complex systems.

The difficulty in exploring complex systems appears when attempting to explore their behavior spaces. One often wants to discover singularities and to explain the sources of them. In essence, it is desirable to construct a map from a considered system outcome space to its parameter space. Here, characterizations of interference constitute the outcome space, and entry programs constitute the parameter space. Thus, one wants to discover all interference mechanisms and their corresponding programs. Complex systems suffer from butterfly effects, which means that

two close initial conditions do not necessarily induce close outcomes. Such systems also suffer from attractor effects, that is, a tendency of the system to evolve towards a set of states, and stochasticity.

Various methods have been attempted to identify mechanisms that are sources of interference, often too complex, and therefore unsuited to industrial-case scenario [7, 4, 5]. Thus we believe that our automated discovery algorithms can be very useful to characterize their behavior. We have developed a prototype successfully showing how such automated discovery algorithms can discover a great diversity of points in a simplified outcome space.

Diversity-driven algorithms are mostly online algorithms. Indeed, our case of hardware interference identification requires performing executions while performing exploration. We choose to employ a population-based achievement strategy which enable dealing with a low experimental budget. Our sample efficient exploration model makes use of basic code and we choose to not involve any learning process.

In the following, we describe fundamental aspects of the exploration, that is, how to interact our complex system. Therefore we define a parameter space and a behavior space for the discovery agent.

Objectives

The purpose of the study is to tackle the interference general problem on multi-core platforms with the automated discovery framework:

- Demonstrate the ability of automated discovery algorithms to interact with multi-core platforms by inducing a great variety of interference features in a space made of performance counters.
- Discuss how AD algorithms can be used to the further objective of identify micro architectural mechanisms that can lead to the occurrence of an interference.

In order to provide a proof of concept for applying automated discovery framework on the problem of interference identification, one choose to explore a simplified simulated model of dual-core architecture. We aim to demonstrate that outcomes that are hard to observe with either random strategy or human guided strategy can be discovered by an autonomous agent within a exploration strategy guided by curiosity. Furthermore, we would like to discover non-trivial interference mechanisms leading to interference. In the following, we provide explanations for the phenomenon of interference, and we present our simulated simplified model upon which we will demonstrate our automated exploration algorithm.

2.1 Motivation

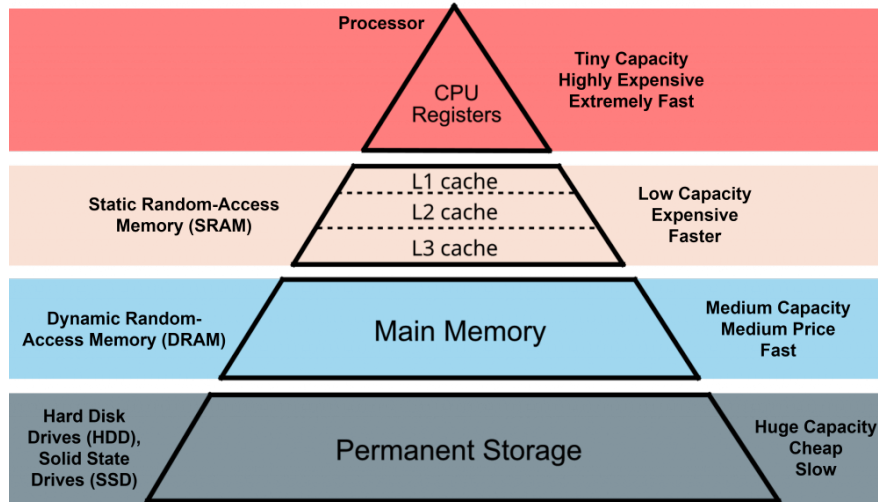


Figure 1: Scheme of a common computer memory hierarchy [8]

Systems of chips face a general trade-off between the necessary time to access information stored in a memory component and their storage capacity. As an element is closer to the CPU, its accessing time and storage capacity are smaller. This principle is illustrated in figure 1. Because they are directly on the CPU, registers typically serve information almost instantly. Their storage capacity is very small, and only essential information is stored. When necessary, information is stored in the cache which has larger latencies but a higher storage capacity than the registers. Information that is less frequently used is stored in the main memory. The main memory consists of DRAM cells that offer a great storage capacity. However, the main memory often operates orders of magnitude slower than the CPU and is also considerably slower than the cache. Therefore, there may be a noticeable slowdown each time the CPU needs to access the main memory. As a consequence of organization, competition for accessing stored data and instructions within shared caches and shared memory between CPU cores raises the execution time, and thus create potential interference channels.

Definition 2.1: Interference

An interference is a phenomenon such that, for identical initial conditions, the execution time of an application S_1 running in isolation ($S_1, _$) on a platform differs from the execution time of S_1 running with an application S_2 on the platform. (S_1, S_2).

Interference arises from contentions on the shared resources, e.g. when multiple cores compete for allocating information in cache or memory. Another case is the competition for accessing a shared bus resulting in bandwidth limitations and delays in data transfer. To summarize, we can distinguish distinct categories of interferences occurring because of contentions on shared resources:

- **Cache contention:** When multiple cores compete for space in shared caches, evicting each other's data and causing cache misses.
- **Memory contention:** When multiple cores access a shared memory simultaneously (such as the main memory), leading to a bottleneck due to limited memory bandwidth.
- **Bus contention:** When cores compete for access to a shared bus, resulting in bandwidth limitations and delays in data transfer.
- **I/O Contention:** When multiple processes try to perform I/O operations at the same time, leading to delays and reduced I/O performance.

As there are multiple types of interference, there exists various type of interference sources to discover. In the following, we will briefly present existing methods that tackle real-time system challenges.

2.2 Related works

The interest in AI techniques within the real-time community is relatively recent. Although these studies do not always focus strictly on interferences, we can mention, for example, research on estimating worst-case execution times (in both single-core and multi-core contexts), on estimating overheads due to interferences, and automatically constructing efficient interferers for programs deployed in multi-core environments. Although studies tackling interference on embedded platforms using AI-based techniques exist, to our knowledge, no studies have explicitly targeted their use for identifying interferences. Nevertheless, we will review the key works in these related areas and explore how they might be adapted or inspire solutions for our problem.

Several works have focused on the use of Machine Learning to estimate Worst-Case Execution Times, being for single-core platforms [1] or multi-core ones [3].

A different direction from ML-based estimators is to build interfering environments. An interesting work that is closest from our objective is the Kryptonite approach [9, 10] which introduces a framework that synthesizes a maximally interfering, or Worst-Case Program Interference (WCPI) environment for a Program Under Test (PUT) executing on multi-core hardware. The approach relies on a set of code snippets, called gadgets, to hammer specific shared resources in the multi-core platform and create interferences. Kryptonite then arranges the gadgets in order to maximize interferences in two phases: firstly, a greedy approach is used to iteratively build a sequence of gadgets that increases execution times of the PUT. And secondly, a Reinforcement Learning (RL) algorithm is used to fine-tune the interfering environment. Although the objective is not the same, these approaches are motivating, as a complex system is manipulated, and the objective of reaching an optimum in an output space by exploring a code space is fulfilled. As opposed to such an optimization method that minimizes a criterium, we propose a space coverage technique to induce a large diversity of behaviors of a given complex system. In the next section, we present our basic simulator upon which we will perform our exploration.

2.3 Description of our dual-core model

We justify the implementation of the simulator and briefly describe components and their behaviors. A minimum of two cores with shared memory components is by definition necessary to induce interference phenomena. Thus, our basic simulator model contains:

- two cores,
- their respective private L1 cache
- a shared L2 cache unit and a DDR memory with its controller.
- An interconnect model that transmits instructions sent by the cache to the memory.

With such a configuration, interference arises in shared components, such as the cache L2, the DDR memory controller, and the interconnect. In the following, we explain the functioning of some of the basic blocks of our simulator.

2.3.1 Core model

As interference is mostly due to memory mechanisms, the simulation of cores handles exclusively memory accesses instructions, that is, the instruction set is reduced to read and write operations in memory since, ultimately, only memory accesses are simulated. In the following, we denote the "read" operations by RD and the "write" operations by WR.

In our algorithms, for each core, a simple loop models the "fetch and execute" cycle. At each CPU cycle, the core can :

1. wait for the end of an RD operation,

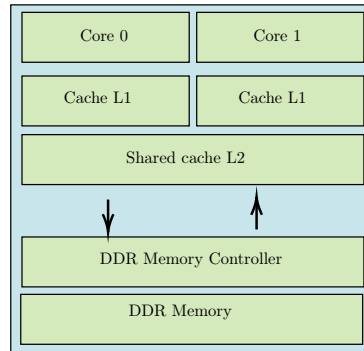


Figure 2: Scheme of our dual core architecture. Each core has a private L1 cache and shares a L2 cache with the other one. The L2 cache is connected to the main DDR memory via its DDR memory controller.

2. execute an RD,
3. execute a WR,
4. execute an instruction that does not perform memory accesses, which, in the case of the simulator, amounts to doing nothing.

As in real-life applications, we make use of a set of virtual addresses that maps to the set of physical addresses, that is, elements of the micro-architecture memory. Within our programs, virtual addresses are simply modeled as numbers. The function that associates a virtual address to a location in the DDR memory is given in A.4.

Within the pairs, the programs do not exchange data to be written or read, are to be executable and interpretable, moreover, this data is not modeled as it does not play any role in interference phenomena as opposed to instructions. Programs consist of RD and WR at given addresses. The shipment is done via the private L1 caches as instructions are not stored neither in the cache nor in the memory. Thus, programs are concretized as simple dictionaries with format {cycle:(type,address)} as follows. For each instruction in a program, its key is the cycle at which the considered instruction is sent to the simplified simulator.

```
program_core_0 = {23:('read', 11),37:('read',17),49:('write',6)}
program_core_1 = {12:('write', 24),18('read',39),32:('write',37)}
```

The maximum length of a program and the largest cycle for which an instruction is sent are fixed as parameters. This model is obviously very simplified compared to the actual operation of a physical target device. Waiting for the completion of a read memory access to continue the code execution does not correspond to reality because the processor has various mechanisms precisely allowing to hide these latencies. This wait is only necessary to preserve the true enter access dependencies (e.g., a program WR @x RD @x must be executed in this order and be preserved at run time to maintain program semantics).

2.3.2 Cache model

Each cache level is configurable in total size, cache line size and associativity. By default, the behavior is of type "write back", that is, writing in lower-level memory occurs when a cache line is evicted. During the write operation, the line is simply marked as "dirty" and is written in the lower level memory during its eviction. It is also possible to employ a "write-through" behavior in which writing into the lower-level memory is done immediately. The management of the eviction of cache lines is carried out by a PLRU whose role is to determine the line to be replaced based on the current state of the cache. Ideally, we would like to implement a behavior of the LRU (Least Recently Used) type, which would consist in eliminating the line used less recently in order to make the most of the locality principle¹. However, this strategy is expensive to implement and we often prefer to use a simpler mechanism called Pseudo-LRU which relies on a binary tree.

¹According to the spatial locality principle, a data is very likely to be accessed when one of its neighbors has recently been accessed, whereas according to temporal locality principle, a recently accessed data is very likely to be accessed again.[5]

Thus, this algorithm includes a function that maintains the data structure with a binary tree and which allows to choose the next row to be evicted based on memory accesses. It also includes a function allowing to choose the next cache row to be evicted using the information contained in the binary tree. The last level cache (L2 in figure 4) is shared by both memory hierarchies. There is no cache coherency management mechanism.

2.3.3 DDR model

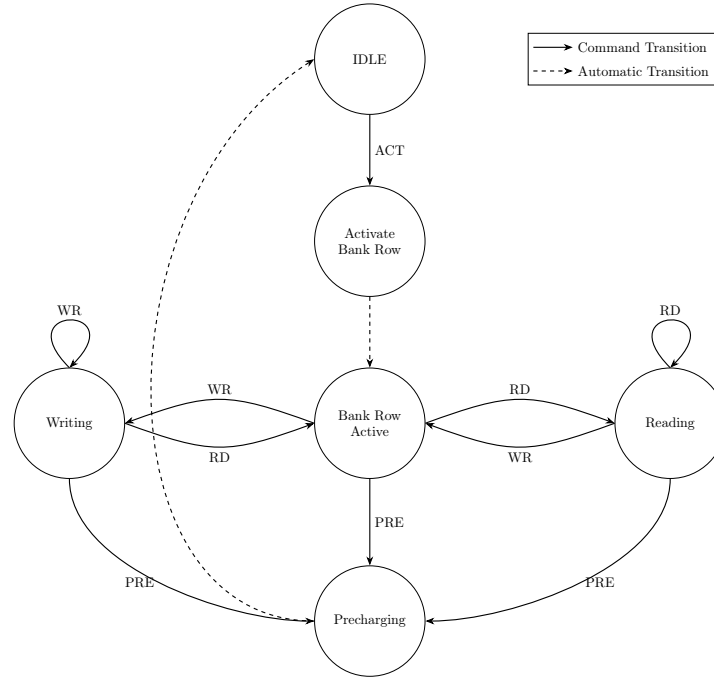


Figure 3: Simplified state diagram assuming a single bank [7]

Let's recall that DRAM is organized in rows and columns banks. Each bank contains an additional row called row buffer. When an information is accessed, its row is stored in the row buffer.

In our model, the memory consists of several banks, each of which contains a row buffer acting as a cache. Latencies are modelled and play an important role for the interference delay in the following way:

- Access to different rows from the same bank induces important delays because of the temporal cost for changing rows in the buffer.
- Simultaneous accesses to distinct rows from the same bank induce larger temporal costs than simultaneous accesses to distinct banks. In other words, intra-bank interference induces larger delays than inter-bank interference.

Each bank is managed by a timed state machine that reproduces the one described in [7] (see figure 2.3.3). The state of the banks of the DDR determines the completion dates of operations. Requests are issued by the DDR controller. Let's note PRE, the operation that deactivates an open row of a given bank, this action writes the selected information from the row buffer to the memory. And ACT the operation that activates a closed row, by storing the information from the given row in the buffer. Additional informations on the DDR are given in A.

2.3.4 DDR controller model

The memory controller is in charge of reordering memory access requests to maximize access rate memory. It implements several queues: a queue for commands, a queue for read data, a queue for the data in writing. Prioritization rules are modeled. Indeed, the RD are prioritized on the WR in order to minimize waiting times, but

while preserving memory consistency (WR @x RD @x must be executed in this order). Instructions targeting lines that are already in row buffers are prioritized in order to avoid the penalty related to completing a PRE=>ACT sequence. RD and WR are processed in "batch" to avoid the overhead related to the transition between RD and WR. Mechanisms of our DDR controller are described in A.2.

2.4 Overview of the exploration strategy

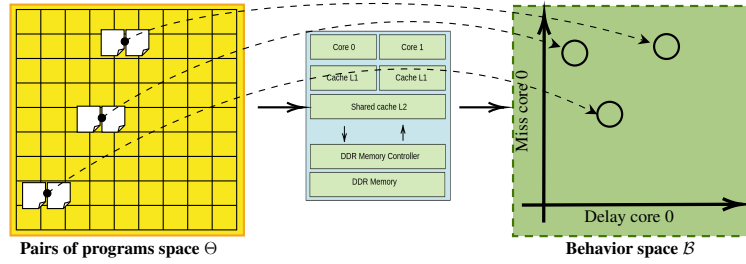


Figure 4: Illustration of a complex system associated with a multi-core hardware. On the left side of the figure, pairs of simplified assembly programs are represented in the parameter space Θ . Each of them is executed on the platform, and corresponds to an outcome (behavior) on the right hand side, representing the behavior space $\mathcal{B}$. In our case, both the parameter space and the behavior space can be large dimension spaces. Although in general, the parameter space has a larger dimension than the behavior space.

The Intrinsically Motivated Goal Exploration Process is an instance of automated discovery algorithms. This diversity-driven strategy aims to maximally cover a behavior space. During its exploration, a discovery agent sequentially selects parameters leading to the highest diversity in that space. For achieving this, the agent iteratively samples goals uniformly in the behavior space, and generates appropriate parameters to reach them. The exploration is made of several episodes called "experiments", during which the agent continually improves its ability to discover new singularities. For example, the study [1] chooses the chains of amino acids to constitute a parameter space and 3D shapes of the resulting protein to constitute the behavior space.

For interference identification, the considered complex system is the simulated hardware platform. In contrast to Kryptonite for example, which attempts to minimize/maximize interference delays, an IMGEP instead aims to discover a large diversity of possible interference patterns. We wish to explore a behavior space $\mathcal{B}$ that characterizes interference patterns within the hardware by manipulating inputs from a parameter space Θ , see figure 4. In our case, we establish a parameter space made of simplified assembly programs. It is essential, on the one hand, to formally model basic elements of the exploration, such as the parameter space Θ and the behavior space $\mathcal{B}$, and on the other hand, to specify the agent's internal models.

Definition 2.2: Pair of programs

We call "Pair of programs", a couple of two programs of code S_1 and S_2 allocated respectively to the C_1 and C_2 cores of an execution platform. S_1 and S_2 are independent code programs. In particular, they do not exchange data.

Definition 2.3: Experiment

We call "experiment", the interaction with our complex system. It involves parallel execution of the programs S_1 and S_2 , and the execution of S_1 and S_2 alone. All experiments are to be independent, this involves for example to clear the cache, ddr and interconnect before every execution.

From an IMGEP perspective, an element of a behavior space is seen as a goal to achieve. The discovery agent samples goals to solve and uses its strategy to solve them. Two internal models are used for these respective aspects, one for goal generation strategy $\mathcal{G}$, and a goal strategy achievement model Π . As illustrated on the figure 5, the

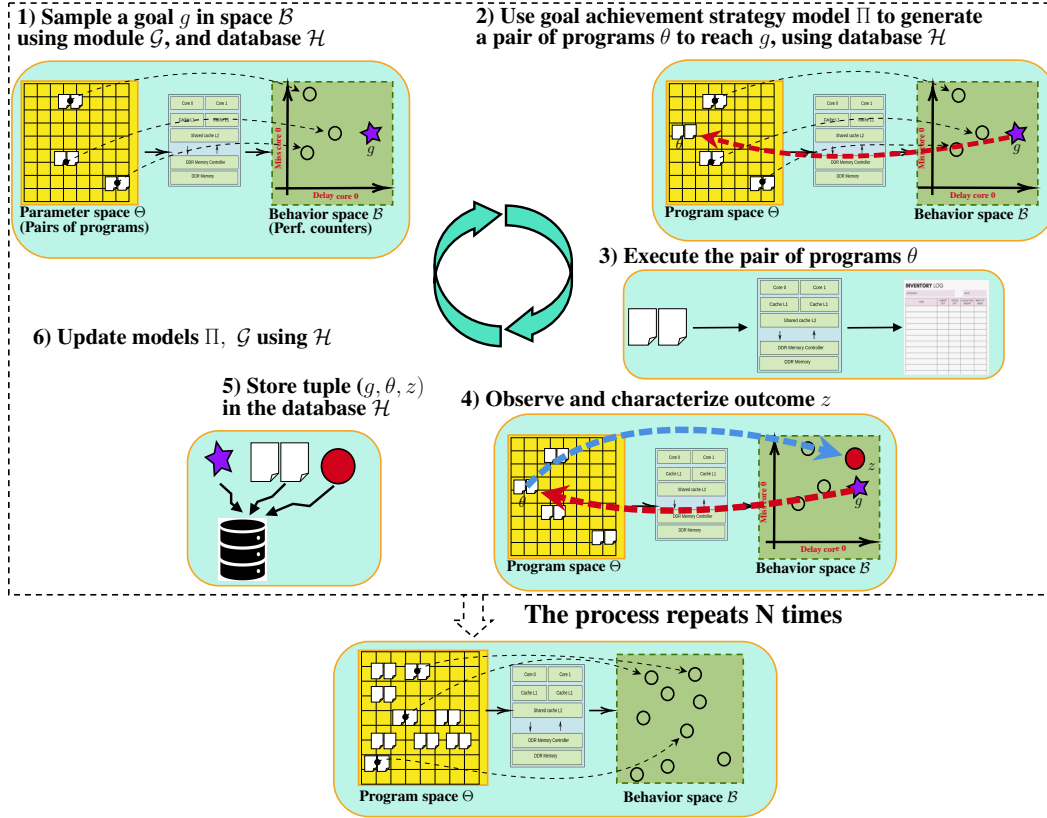


Figure 5: Schematic view of an IMGEP for our case of interference identification, based on the general description of IMGEP B.1.

agent fills a database $\mathcal{H}$ to update its internal models during the exploration, and thus the acquired data is reused to extract potential solutions to solve other goals.

An IMGEP is a loop algorithm that iterates online by interacting with the considered complex system. The algorithm starts with a warming set of N_{init} iterations. This helps the exploration to escape poor knowledge regions.

This type of architecture highlights previous independent experiments stored in the database $\mathcal{H}$. Using efficient sampling methods, this algorithm allows one to quickly reach a great diversity of outcomes in the behavior space. Moreover, its usage of the database prevents forgetting during the exploration.

2.5 Parameter space

A pair of programs is required for running each experiment. As presented on figure 5, the discovery agent generates a pair $\theta \in \Theta$ at each iteration. Following the general program constraints discussed in part 2.3.1, we remind that most sources of interference are due to memory mechanisms, therefore we choose to produce pairs of simplified assembly instructions for constituting a parameter space Θ . Indeed, as presented before, only memory instructions RD and WR are modeled. For getting closer to a real-life application, we can model two distinct set of addresses for the cores, i.e core 0 :0,20 and core 1: 21,40. As two distinct addresses can map to some same physical addresses, it will be still possible to observe interference.

With the maximum program lenght and largest shipment cycle being carefully chosen, we synthetize a parameter space of a reasonable cardinal, for thus having a better understanding of the results.

2.6 Behavior space

All microarchitectural mechanisms known as the simulator is a white box. We wish to identify the ones responsible for interferences. A set of relevant performance counters provide building blocks for the behavior space, see table 1.

Performance counter	Category	Description
Processor cycles	General	Number of executed cycles
Instruction completed	General	Number of completed instructions
Decode stalled	General	Number of cycles in a waiting status
Cache misses	Cache	Number of cache miss L1/L2
Cache store allocates	Cache	Number of line allocations in L1/L2
Cache demand access	Cache	Number of requests for L1/L2
DDR store misses	DDR	Number of stores DDR
DDR load misses	DDR	Number of miss
DDR demand access	DDR	Number of requests
DDR store allocates	DDR	Number of line allocations

Table 1: Set of available performance counter in the dual-core simulator

Such performance counters can be clock cycles, row misses, instruction types, branch mispredictions, and number of stalls. Indeed the phenomena quantified by those performance counters allow the description of conflicts occurring in shared resources and thus are directly related with interferences. In general, information on the platform will always be needed in a context where interference is being addressed. If a platform is black box, all the requirements of the certification file cannot be fulfilled.

Each element of the established behavior space is perceived as a potential goal for the discovery agent. A goal is defined as a pair $g = (z_g, \mathcal{L}_g)$ with an embedding $z_g \in \mathcal{B}$ and an evaluation function $\mathcal{L}_g = \mathcal{D}(\cdot, z_g)$. Ideally, $\mathcal{B}$ has "good properties" such as being a low-dimensional metric space. The metric property is important for the goal achievement strategy of the discovery agent. Indeed, one wants to evaluate the distances between some previous experiment outcomes z and a goal embedding z_g . For example: $\mathcal{L}_g(z) = \|z - z_g\|_2$. This aspect is essential for designing an efficient goal achievement strategy, as one wants to efficiently select previous experiment outcomes and their associated parameters that are potentially suited for the generation of new candidate parameters.

Features	Category	Description
Processor cycles diff core	General	Difference $[\text{nb of executed cycles}]_{\text{non-iso}} - [\text{nb of executed cycles}]_{\text{iso.}}$
Processor mutual	General	Difference $[\text{nb of executed cycles}]_{\text{non-iso. core 0}} - [\text{nb of executed cycles}]_{\text{non-iso core 1.}}$
Cache write misses	L2 Cache	Difference $[\text{nb of L2 cache miss}]_{\text{non-iso}} - [\text{nb of L2 cache miss}]_{\text{iso.}}$
Cache write hits	L2 Cache	Difference $[\text{nb of L2 cache miss}]_{\text{non-iso}} - [\text{nb of L2 cache miss}]_{\text{iso.}}$
Cache reads misses	L2 Cache	Difference $[\text{nb of L2 cache miss}]_{\text{non-iso}} - [\text{nb of L2 cache miss}]_{\text{iso.}}$
Cache reads hits	L2 Cache	Difference $[\text{nb of L2 cache miss}]_{\text{non-iso}} - [\text{nb of L2 cache miss}]_{\text{iso.}}$
DDR misses	DDR	Difference $[\text{nb of misses}]_{\text{non-iso}} - [\text{nb of misses}]_{\text{iso.}}$ for each pair (bank,row)
DDR hits	DDR	Difference $[\text{nb of hits}]_{\text{non-iso}} - [\text{nb of hits}]_{\text{iso.}}$ for each pair (bank,row)

Table 2: Features constituting the behavior space $\mathcal{B}$. For any performance counter C , we calculate two features F_C , as non-iso. vs iso. differences $F_C \equiv C[\text{non-iso}] - C[\text{iso.}]$. "iso." corresponds to an execution with only one of the two cores, 0 or 1.

Because interferences arise following micro-architectural behavior conflicts, we target shared resources performance counter differences in values between the executions in isolation (**iso**) and the executions in non isolation (**non-iso**). Given a pair programs, we calculate for each selected performance counter its difference when executing in isolation on one core and in non-isolation on both cores. The features in the table 2 constitute the behavior space $\mathcal{B}$.

With n_{banks} banks and n_{rows} rows, the dimension of the $\mathcal{B}$ is given by $n = 3 + 4 \cdot 2 + 2 \cdot 2 \cdot n_{\text{banks}} \cdot n_{\text{rows}}$ and $\mathcal{B} \subset \mathbb{R}^n$. If $n_{\text{row}} = 3$ and $n_{\text{banks}} = 4$, we then have $\mathcal{B} \subset \mathbb{R}^{59}$. The performance of the exploration algorithm might decrease in case of a large behavior space dimension, as a consequence of the curse of dimensionality when the interpretation of distances loses its sense. Therefore, working with a parsimonious set of data might imply better results in terms of diversity measures. This chosen configuration of a behavior space is an Euclidean space.

Therefore, we can measure distances between the current goal embedding z_g and known features z in our database. Let's remark that the performance counters associated with the features presented in table 2 are not systematically available when working with a material device.

2.7 Goal sampling

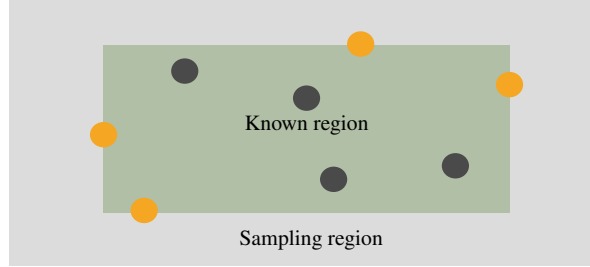


Figure 6: Schematic view of a sampling process. Yellow points represent border outcomes from the already known region and allow determining the sampling region for the next experiments.

For the discovery agent, the concept of "curiosity" is materialized by the sampling of goals. For the sampling of goals, the simplest case scenario is to sample them uniformly at random in a connex subregion of $\mathcal{B}$ including the set of already discovered points. As illustrated in figure 6, choosing the sampling region slightly larger than a subregion including the set of already discovered points allows to target regions of $\mathcal{G}$ that have not been reached yet but are close to the already discovered ones.

We periodically set the sampling boundaries based on the database $\mathcal{H}$ and we simply sample goals with a uniform distribution. For instance, we determine the minima and maxima of the discovered values for each 1-dimensional feature:

- $\min_{\mathcal{B}g} := (\min\{z_1, z \in \mathcal{H}\}, \dots, \min\{z_{\dim(\mathcal{B})}, z \in \mathcal{H}\})$
- $\max_{\mathcal{B}g} := (\max\{z_1, z \in \mathcal{H}\}, \dots, \max\{z_{\dim(\mathcal{B})}, z \in \mathcal{H}\})$

Then, to establish the slightly larger sampling region, we fix two factors e.g $f_1 = 0.8, f_2 = 1.2$, that determine percentages of the previous minimal and maximal values for the new uniform distribution that will be used to sample goals $g \in \mathcal{B}$. Since $\mathcal{G}$ is a multidimensional space, the resulting distribution will be a product of several 1-dimensional uniform distributions:

$$z_g \sim \mathcal{U}([f_1 \cdot (\min_{\mathcal{B}g})_1, f_2 \cdot (\min_{\mathcal{B}g})_2]) \otimes \dots \otimes \mathcal{U}([f_1 \cdot (\min_{\mathcal{B}g})_{\dim(\mathcal{B})}, f_2 \cdot (\max_{\mathcal{B}g})_{\dim(\mathcal{B})}])$$

2.8 Goal achievement strategy

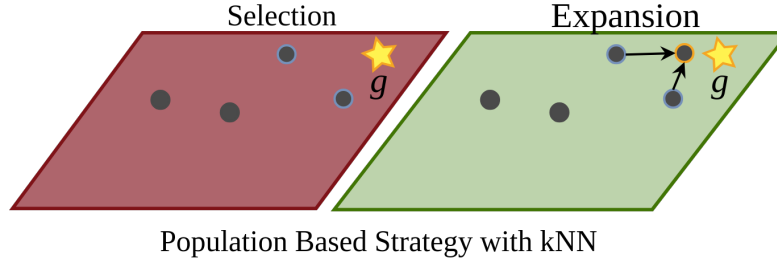


Figure 7: Illustration scheme of a goal achievement strategy. The left hand side illustrates the selection of outcomes stored in the database that will allow expansion, that is the synthesis of a new outcome closer from g (right hand side). Here, the two closest neighbors to g in the database $\mathcal{H}$ are selected, and their parameters are used to obtain the new outcome closer to g .

We present our population based achievement strategy, which resembles an evolutionary algorithm, where the set of pair instructions programs together with their respective outcomes is seen as a population. Thus, our algorithm implies the performance of mutations and of reproductions of the individuals.

At each step during an IMGEP exploration, the programs (parameters) and outcomes $((S_1, S_2), z) \in \Theta \times \mathcal{B}$ that make up the agent discoveries are stored in the database $\mathcal{H}$. Because the **exploration budget** is low, we choose the agent to perform non-parametric learning, which means that the complexity of the model grows linearly with the size of the database $\mathcal{H}$.

The goal achievement strategy is typically made of two steps with two respective operators. During the **selection** step, the discovery agent selects outcomes. The **expansion** step consists of using their corresponding parameters to produce another one, leading to an outcome close to the goal g , see figure 7.

We use a kNN ² model as a selection operator. This selection operator relies on an expansion operator, as kNN **selects** k promising outcomes from $\mathcal{H}$ and **expands** it by producing a candidate pair of programs to achieve g . In particular, we select the k closest outcomes to the given goal g within the database $\mathcal{H}$ using a well designed evaluation function $\mathcal{L}$. We then mix the k corresponding pairs of programs together to obtain a candidate pair of programs to achieve goal g .

We establish the weighted evaluation function so that all features are equally considered in the selection step:

$$\mathcal{L}_g(z) = \sum_F \frac{1}{\max_F - \min_F} \cdot (z_F - g)^2, \forall z, g \in \mathcal{B}$$

For mixing the programs, we select contiguous parts from multiple programs, preserving timing, and ensuring the resulting program fits within the fixed maximum cycle. The length of each segment is chosen at random. The key parameters are the **numbers of segments to produce** the output program and the **number of programs to mix**.

For ensuring more efficiency, one also apply a **mutation operator** following the mixing operator. The mutations performed are : 1. randomly change existing instructions, 2. Delete random instructions, 3. add random instructions. Consequently, different patterns of the programs change, as the number of instructions, number of memory accesses, and the set of used addresses.

In the particular case of $k = 1$, we simply select the closest observation z of g in the database $\mathcal{H}$ and apply the **mutation operator** to obtain the candidate parameter. The mixing and mutation operators are detailed in B.2 and B.3

During the first iterations of the algorithm, IMGEP typically outputs outcomes that are not diverse. To overcome the difficulty of the agent of exploring during its first iterations, a warm up made of several random iterations is

²Let's recall that kNN is a supervised machine learning model that predicts a label y of a given vector x , using the k closest neighbors of x associated labels.

performed. The agent initially induces a poor diversity, and while the population stored in the database $\mathcal{H}$ grows, the agent is more likely to solve its goals. Hence, on average over time, goals targeted by the agent are not reached with the parameters θ outputted by IMGEP, meanwhile, it is a powerful method that leads to high-diversity discoveries.

3 Results

We run IMGEP with $n_{\text{rows}} = 3$, $n_{\text{banks}} = 4$, with $N = 10000$ iterations including a warming set of $N_{\text{init}} = 1000$ iterations of random program synthesis, and with $k = 1, 2$ and 3 as parameter for the kNN model of the goal achievement strategy. Each program contains between 1 and 10 instructions. The maximum cycle for the last instruction to be sent is 60. As said previously, we model two distinct set of addresses for the cores, i.e core 0 :0,20 and core 1: 21,40. The selection of the simulator parameters chosen to run our experiments is available in A.4.

We first assess the diversity of the resulting datasets, through diversity measures. Secondly, we explore the results and attempt to infer diagnostics from the data. Finally, we discuss approaches that would be more relevant for IMGEP.

3.1 Diversity evaluation

3.1.1 Method

We compare our method with two simple strategies: 1. a random exploration, that is, generating pairs of programs randomly. And 2. to be compared with IMGEP for k , an algorithm that selects k pairs of programs randomly from its database $\mathcal{H}$, mixes them to obtain a single pair, and mutate it. Indeed, we would like to ensure that a larger IMGEP diversity does not occur simply because of the design of the mixing and mutation operators. For this matter, we aim to demonstrate that mixing and mutating random selections of programs from the same warming set does not lead to a significantly higher diversity than that of IMGEP runs. The mixing and mutation operators are the same as IMGEP's. Hence, for a fair comparison, the first N_{init} iterations of the random exploration are shared by the IMGEP runs and by the operator combination exploration. We quantitatively assess each exploration process, by simply establishing diversity measurements of the outcomes stored in the respective database $\mathcal{H}$. In particular, we observe how the behavior space $\mathcal{B}$ is covered by the corresponding outcomes $o_1, \dots, o_n$.

In the following we denote two particular subspaces of the behavior space by **time behavior space** and **hit/miss space**. We construct the following measure suitable to assess diversity: The number of bins filled in a n -dimensional histogram. With $n = 3 + 4 \cdot 2 + 2 \cdot 2 \cdot n_{\text{banks}} \cdot n_{\text{rows}}$, being the dimension of each outcome we obtain after an experiment, see part 2.6. We denote the quantity corresponding to each axis j of an outcome F by $F_j \in \mathbb{R}$, $1 \leq j \leq n$. The values of $\inf(F_j)$ and $\sup(F_j)$ are carefully chosen so that $\forall F, F' \in]\inf(F_j), \sup(F_j)[$.

Let us denote a chosen partition of $] \inf(F_j), \sup(F_j)[$ by $x(j)$, $1 \leq j \leq n$: $\inf(F_j) = x(j)_0 < \dots < x(j)_{n_j} = \sup(F_j)$. For a family of partitions $(x(1), \dots, x(n))$, we define a bin as :

$$\text{bin}((x(1)_{k_1}, \dots, x(n)_{k_n})) = [x(1)_{k_0}, x(1)_{k_0+1}[\times \dots \times [x(n)_{k_n}, x(n)_{k_n+1}[, \forall k_0 \in \mathbb{N} \cup [0, n_0[, \dots, \forall k_n \in \mathbb{N} \cup [0, n_n[\quad (1)$$

We define $\mathcal{D}_{(x(1), \dots, x(n))}$, the positive measure of data clouds associated with the set of all bins with the partitions $x(j)$. For a set of points discovered H from $\mathcal{H}$, we have:

$$\mathcal{D}_{(x(1), \dots, x(n))}(H) = \sum_{(k_0, \dots, k_n)} \mathbf{1}_{\{\text{card}(\text{bin}(x(1)_{k_1}, \dots, x(n)_{k_n}) \cap H) > 0\}} \quad (2)$$

We choose a partition such that $|x(j)_{k_j} - x(j)_{k_j+1}| = 5, \forall 1 \leq j \leq n, \forall k_j$. $\mathcal{D}_{(x(1), \dots, x(n))}$ is a well defined positive measure for the associated set of non-empty bins of a data cloud H .

3.1.2 Diversity analysis

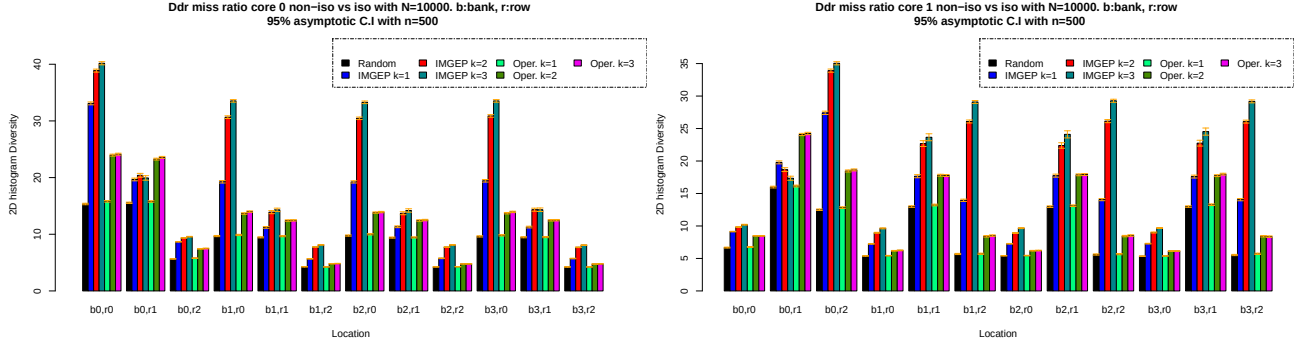


Figure 8: Diversity of miss ratio for both cores and all bank row pairs. Orange bars represent lower and upper confidence bounds using the Student's t-distribution asymptotic confidence interval $\mathbb{P}(\bar{X} - u_{1-\frac{1}{\alpha}} \sqrt{\frac{\hat{v}}{n}} \leq g(\theta) \leq \bar{X} + u_{1-\frac{1}{\alpha}} \sqrt{\frac{\hat{v}}{n}}) \rightarrow .95$ with mean and unbiased standard deviation estimators.

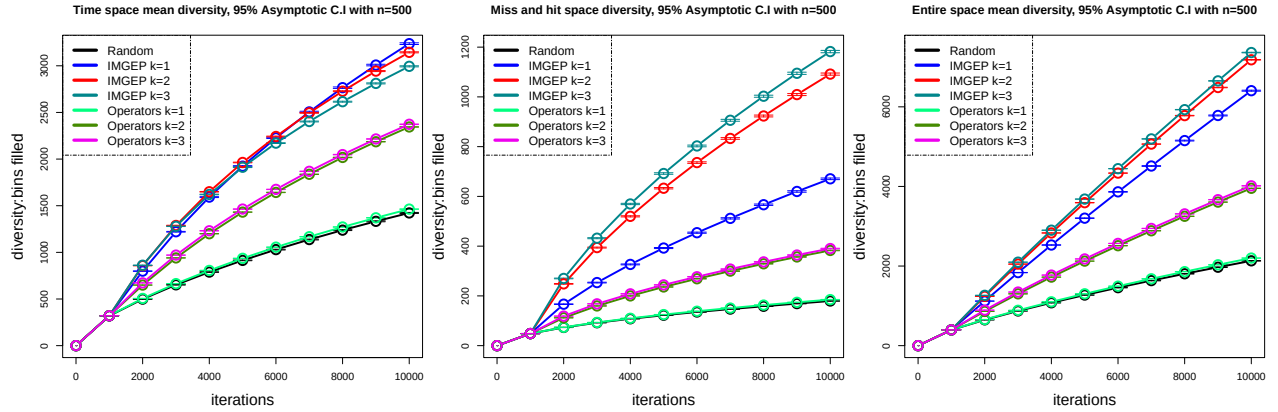


Figure 9: On the two left plots, we visualize the diversity on the time behavior space and on hit/miss space, that is, how the number of bins filled in a multidimensional histogram increases during exploration. On the right plot, we visualize the diversity of the entire dataset, the dimension of the considered histogram is the number of features (59). Results of $N = 10^3$ iterations with a warming set of $N_{init} = 1000$ iterations of random program synthesis

For each value of k , which is the number of neighbors in the optimization policy achievement model, we show that IMGEp leads to a higher diversity than the operator combination exploration. Indeed, IMGEp finds many more distinct values of miss ratios across the distinct locations—pairs (bank, row)—in the main memory, see figure 8. Note that the discoveries of "miss" occurrences probably helps to induce more interferences. Indeed, whenever a requested data is missing in a DDR buffer, it has to be recharged, and this involves an additional delay. Moreover, the figure 9 shows that the IMGEp diversity measurement increases faster on the entire behavior space than that of the operator combination exploration, and simultaneously on the distinct L2 & DDR miss/hit, and execution time goal subspaces.

Generating programs with IMGEp allows to observe much more interference than generating programs randomly. Interference occurs whenever the execution time in isolation differs from the execution time in non-isolation. To some extend, we consider that a pair of programs produces an interference if $|\text{time}[\text{non-isolation}] - \text{time}[\text{isolation}]| > 0$. In the example run presented on figure 10, the bars with axis 0 on histograms for values of $|\text{time}[\text{non-isolation}] - \text{time}[\text{isolation}]|$ demonstrate that exploring the simulator with randomly generated programs produces interference about 4 times out of 10 whereas about 7 times out of 10 for IMGEp.

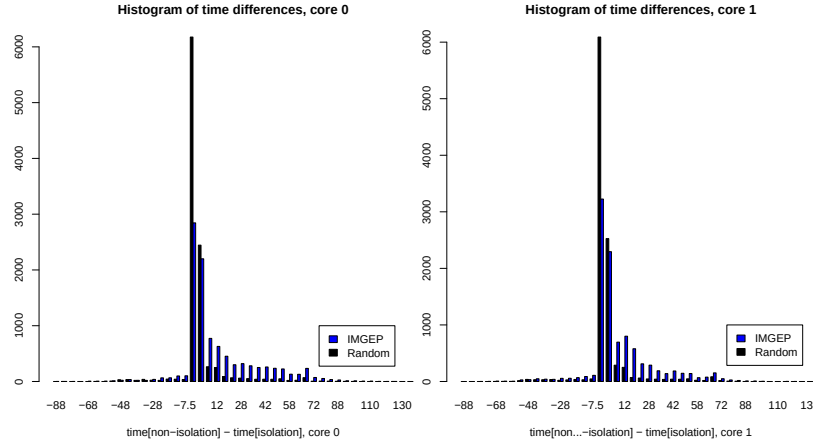


Figure 10: Diversity for execution time behavior space. Results of $N = 10^3$ IMGEP iterations with $k = 1$ and a warming set of $N_{init} = 1000$ iterations of random program synthesis.

Information on the execution time is represented for each program on figure 11. More precisely, we attribute to each pair of programs executed simultaneously on core 0 and core 1 two respective points on the distinct scatter plots to represent its couple ($\text{time}[\text{isolation}], \text{time}[\text{non-isolation}]$). This representation allows us to spot interference, that is, whenever points are not located on the red identity line. Points above the red line represent programs that are slowed down by the parallel execution on the other core, whereas points below the line represent programs which are accelerated, which means the execution time is faster when the program is executed together with another program. This is due to the absence of coherency management mechanism. For instance, the following pair of programs results in the acceleration of both programs. One can show DDr bank and line numbers instead of addresses. Instructions of core 1 on cycles 36, 40 and 41 are processed earlier when the program on core 0 is executed because of its instructions on cycles 20 and 30 which load the lines {bank 1, row 1} and {bank 0, row 1} in their buffer. Hence, sequences $\text{PRE} \Rightarrow \text{ACT}$ are avoided as the data is already in the buffer.

```
bk_rw_core0 = {20: {'bk': 1, 'rw': 1}, 27: {'bk': 3, 'rw': 0}, 30: {'bk': 0, 'rw': 1}, 31: {'bk': 2, 'rw': 0},
36: {'bk': 1, 'rw': 0}, 43: {'bk': 3, 'rw': 0}, 46: {'bk': 0, 'rw': 0}}
bk_rw_core1 = {6: {'bk': 2, 'rw': 1}, 8: {'bk': 2, 'rw': 1}, 11: {'bk': 3, 'rw': 1}, 18: {'bk': 1, 'rw': 2}, 27: {'bk': 0, 'rw': 2},
36: {'bk': 1, 'rw': 1}, 40: {'bk': 1, 'rw': 1}, 42: {'bk': 0, 'rw': 1}, 45: {'bk': 3, 'rw': 1}, 47: {'bk': 0, 'rw': 2}}
core 1 mutuality 78 cycles, isolation: 162 cycles
core 0 mutuality 108 cycles, isolation: 97 cycles
```

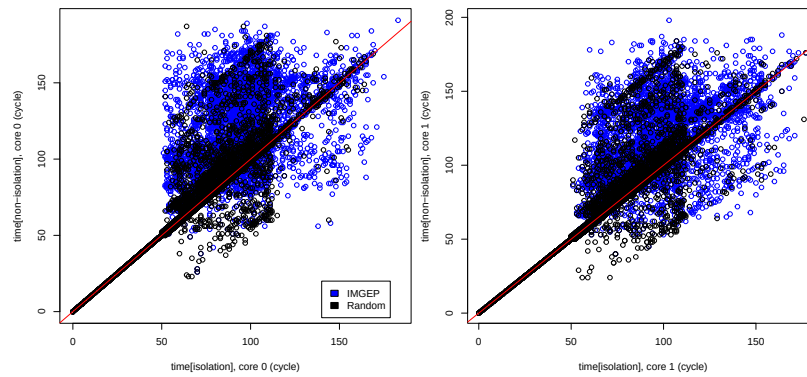


Figure 11: Diversity for execution time behavior space. Results of $N = 10^3$ IMGEP iterations with $k = 1$ and a warming set of $N_{init} = 1000$ iterations of random program synthesis.

In figure 12, we represent values of couples ($\text{time}[\text{isolation}, \text{core } 0] - \text{time}[\text{non-isolation}, \text{core } 0]$, $\text{time}[\text{isolation}, \text{core } 1] - \text{time}[\text{non-isolation}, \text{core } 1]$). Any point with an abscissa or ordinate different than 0 corresponds to a pair a

programs presenting an interference on the core 0 or core 1 respectively. The figure clearly shows that IMGEP cloud spreads more, indicating that our algorithm finds many more interference cases.

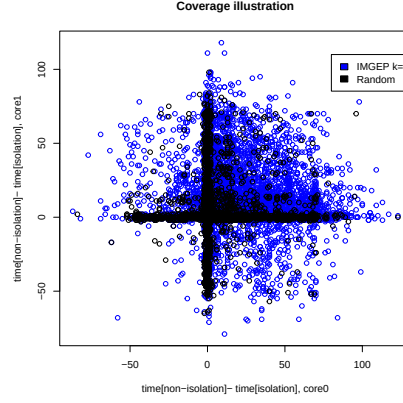


Figure 12: Spanning of IMGEP on time behavior space. Interference occurs on core 0 or 1 when abscissa or ordinate is different than zero. Results of $N = 10^3$ IMGEP iterations with $k = 1$ and a warming set of $N_{init} = 1000$ iterations of random program synthesis.

3.2 Analysis

The results we presented demonstrate through diversity measurements that automated exploration algorithms can cover efficiently an output space of a complex system associated with a simulated hardware model.

The data from which the exploration is performed provides only aggregated information and is not internal. While this implies that characterizing sources of interference with such a defined behavior space is limited, we spot an emerging structure in Figure 13, suggesting that pairs of programs with many requests to the same DDR locations induce a greater variety of interference delays.

Hence, the resulting pairs of programs causing interference can be further analyzed by humans, and our method may be completed to ensure that distinct points in the outcome space provide two distinct sources of interference.

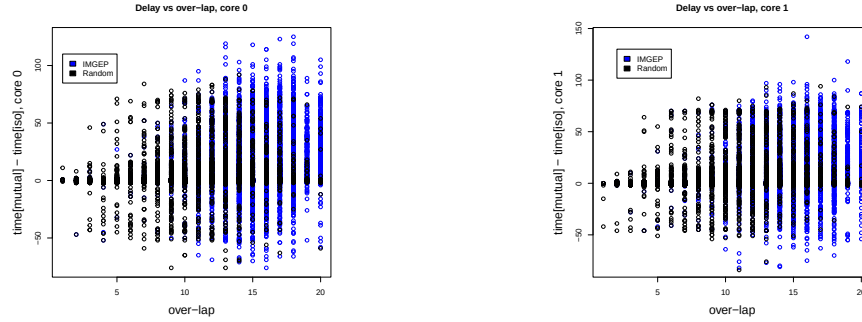


Figure 13: $(\text{time}[\text{non-iso}] - \text{time}[\text{iso}])$ vs non-overlap. We call non-overlap the quantity $f(p_0, p_1) = \sum_j |(\text{nb access row } j, \text{ core } 0) - (\text{nb access row } j, \text{ core } 1)|$. This graph shows the number of access occurring in the same locations for both cores in the main memory is not enough to characterize interference.

We believe that creating a space that fully consists of sources of interference will allow discoveries of non-trivial access patterns. In addition, one wants to discover access patterns that are unknown and difficult to produce by specialized engineers. Furthermore, the resulting behavior space would not necessarily be an Euclidean space, and could, for example, be a graph, a tree, or a semantic space.

4 Conclusion

This work deals with the problem of interacting with SOC's via automated discovery algorithms to help the discovery of singular cases. The method relies on simple machine learning models to structure an autonomous agent that fixes his own goals, and on the establishment of a behavior and a parameter space.

The results demonstrate that automated discovery algorithms can interact successfully with a simplified simulated model of dual-core architecture, and encourage the idea of exploring a real SOC. Indeed, the behavior space, which consists of performance counters, is explored efficiently with IMGEP, as it leads to a higher diversity than exploring the simulator with programs that are randomly generated.

This approach encourages the idea of exploring a behavior space that is more suited for the characterization of mechanisms of interference in order to discover non-trivial sources of interference. In further work, we will aim to develop a strategy that specifically identifies distinct mechanisms that are responsible for interference, rather than exhibiting a large diversity of interferences regardless of the reasons they occur. For this, we will distinguish whether the observation of an interference relies on the activation of a new mechanism. Hence, the objective will be to maximize the number of distinct types of interference.

References

- [1] Abderraouf Nassim Amalou. “Machine learning for timing estimation”. PhD thesis. Université de Rennes, 2023. URL: <https://hal.science/tel-04406029v1>.
- [2] Oudeyer Baranes. “Active Learning of Inverse Models with Intrinsically Motivated Goal Exploration in Robots”. In: *ScienceDirect* (2013). URL: <https://arxiv.org/pdf/1301.4862>.
- [3] Armelle Bonendant and al. “Early WCET Prediction using Machine Learning”. In: *17th International Workshop, on Worst-Case Execution Time Analysis, OpenAccess Series in Informatics* (2017).
- [4] Boniol and al. “PHYLOG certification methodology: a sane way to embed multi-core processors”. In: *10th European Congress on Embedded Real Time Software and Systems, Toulouse, France* (2020). URL: <https://hal.science/hal-02441323/document>.
- [5] Cédric Courtaud. “Caractérisation de la sensibilité aux interférences mémoire dans les systèmes temps-réels embarqués sur des plateformes multi-coeurs”. PhD thesis. Sorbonne Université, 2020.
- [6] Mayalène Etcheverry. “Curiosity-driven AI for Science: Automated Discovery of Self-Organized Structures”. In: *Hal* (2005). URL: <https://mayalenetcheverry.com/>.
- [7] Alfonso Mascareñas González. “DDR SDRAM Interference Minimization via Task and Memory Mapping in a Multi-objective Optimization context on Heterogeneous MPSoCs.Toulouse, ISAE”. In: *Hal* (2022). URL: <https://hal.science/tel-05025151/>.
- [8] Ray Kuldeep. “Memory Hierarchy – How does computer memory work ?” In: (2024). URL: <https://spear-itn.eu/memory-hierarchy-how-does-computer-memory-work/>.
- [9] Nikhilesh Singh and al. “Kryptpnite: Worst-Case Program Interference Estimation on Multi-Core Embedded Systems”. In: *ACM Transactions on Embedded Computing Systems* (2023).
- [10] Nikhilesh Singh and al. “Localizing Program Interference on Multi-core Embedded Systems”. In: *In Proceeding of the 12th European Congress on Embedded Real Time Systems* (2024).

Appendices

A Description of our dual-core model

A.1 Complementary information on the DDR

Memory accesses are done with burst length of 8 bytes.

Our simplified DDR2 model implements the following operations and some are described in A.4

- WR: Write
- RD: Read
- ACT: Activation of a closed row.
- PRE: Deactivation of an open row.
- REF: Refreshment of memory cells.

A.2 Complementary information on the DDR controller

The functioning of both the DDR and the memory controller are coupled. At each cycle:

- The controller determines requests that are achieved, that is, if the current date is superior than the scheduled completion date. In that case, for an RD request, the controller calls a "call back" function which signals to the core that this access is achieved. This callback is used by the core to allow the execution of a new instruction. No callback functionality is not implemented for RD requests.
- The controller treats requests that are located in its entry queue. For each request, it determines whether it can be treated by the DDR according to its state (handled by the DDR state machine) while ensuring minimal delay constraints. It then determines the "best" request to treat depending on a established ranking to prioritize request inducing "row hits", WR over RD requests and arrival order (FIFO policy). The controller finally sends the "best" request to the DDR.

The model is inspired from mechanisms described in [7].

A.3 Interconnect model

The interconnect model can be described as follows:

1. It consists in maintaining a queue (FIFO) of memory requests and in executing request in the arrival order and after a delay corresponding to minimal access latency. A random delay of 2 cycles is added.
2. At each clock cycle, some number of requests in priority is unpiled from the queue among the ones that are executable during the current cycle. The requests are then, sent to the memory. The number of unpiled requests at each cycle is determined by the bus bandwidth and each the counting of the number of treated requests.
3. The requests that could not be treated are moved in the queue of requests that are to be treated in next cycles.
4. The interconnect clock cycle frequency and that of the cores are the same. One could introduce distinct clocks by calling the functions "clock" in our codes at distinct cycles.

A.4 Configuration

This section describes the choice of parameters for our experiments.

Component	Category	Description	Value
Core 0	address range	address range for core 0	0-20
Core 1	address range	address range for core 1	21-40
DDR	bank mapping	Bank mapping for an address a	$a \% n_{banks}$
DDR	row mapping	Row mapping for an address a , each row covers 16 addresses	$a // 16$
DDR	tRCD (cycle)	Delay induced by an ACT operation, that is, the activation of a closed line, by storing its content in the buffer	15
DDR	tRP (cycle)	Delay induced by a PRE operation, that is, the deactivation of an open row for a given bank.	15
DDR	tCAS (cycle)	Delay to access data after a burst for both RD and WR instructions.	15
DDR	tFRC (cycle)	Periodic time for cell refreshments. During this operation, every bank row buffer is precharged	30
DDR	tWR (cycle)	Minimum delay between a WR and a PRE operation	15
DDR	tRTP (cycle)	Minimum delay between an RD and a PRE operation	8
DDR	tCCD (cycle)	Minimum delay between an RD and a WR operation	4
DDR	tRRD (cycle)	Maximum delay between two ACT operations	4
DDR	n_{rows}	Number of rows per bank	3
DDR	n_{banks}	Number of banks in the DDR	4
L1 cache	size	Storage capacity of a cache (bytes)	32
L1 cache	line size	Storage capacity of a line (bytes)	4
L1 cache	associativity	Number of different cache lines each data block can be mapped to.	2
L2 cache	size		512
L2 cache	line size		4
L2 cache	associativity		16
L3 cache	size		512
L3 cache	line size		4
L3 cache	associativity		16

Table 3: Selection of the simulator parameters for our experiments

B Intrinsically Motivated Goal Exploration Process

B.1 Description of IMGEP

A general description of IMGEP can be written as follows:

Algorithm 1 Intrinsically Motivated Goal Exploration Process

Require: Parameter Space Θ , Behavior space $\mathcal{G}$, Representation R
Require: Experimental budget N
Ensure: Initialize behavior space $\mathcal{B}$ and goal generator module $\mathcal{G}$
Ensure: Initialize goal-conditioned optimization policy Π

```

for  $i \leftarrow 1$   $N$  do
  if  $i < N_{init}$  then
    Sample a random parameter  $\theta \sim \mathcal{U}(\Theta)$ 
  else
    Sample target goal  $g \sim \mathcal{G}(\mathcal{H})$ 
    Infer experimental parameters to achieve goal  $\theta \sim \Pi(g, \mathcal{H})$ 
  end if
  Execute experiment with  $\theta$  and observe system outcome  $o$ 
  Characterize behavior  $z = R(o)$ 
  Write  $(\theta, o, z)$  to history  $\mathcal{H}$ 
  Update goal space  $\mathcal{G}$  with  $\mathcal{H}$ 
  Update goal generator module  $\mathcal{G}$  with  $\mathcal{H}$ 
  Update goal conditioned policy optimization  $\Pi$  with  $\mathcal{H}$ 
end for
return  $\mathcal{H}$ 

```

B.2 Mixing operator

We describe below the algorithm designed to mix sequences of simplified assembly algorithms. As a reminder, the algorithms are as follows, that is, dictionaries of form $\{cycle : (type, adress)\}$:

```
program_core_0 = {23:('read', 11),37:('read',17),49:('write',6)}
program_core_1 = {12:('write', 24),18('read',39),32:('write',37)}
```

Algorithm 2 Random Mixing of Instruction Sequences

Require: A list of instruction programs S , number of parts P , random seed s , maximum cycle $C_{\max}$

Ensure: A mixed instruction program M

Initialize random generator with seed s

$Chunks \leftarrow \emptyset$

for all program $p \in S$ **do**

Sort instructions of p by increasing cycle

Split p into P contiguous parts

for all part q of p **do**

Append q to $Chunks$

end for

end for

Randomly shuffle $Chunks$

Concatenate all elements of $Chunks$ into list L

Randomly select $|L|$ distinct cycle values from $[1, C_{\max}]$

Sort selected cycle values in increasing order

for $i \leftarrow 1$ to $|L|$ **do**

Assign cycle $cycles[i]$ to instruction $L[i]$

end for

Construct mixed program M from assigned cycles and instructions

return M

B.3 Mutation operator

Algorithm 3 Mutation operator

// Mutate an instruction sequence by adding, deleting, or modifying instructions.

Require: Instructions θ , Number of mutations N , Maximal cycle c , range for addresses a_{min}, a_{max} , maximal number of instructions i_{max}

Ensure: Initialize a copy θ' of the instructions θ

Ensure: Initialize set of available cycles **freecycles**: **all cycles** - **used cycles**

```

for  $i \leftarrow 1 : N$  do
  if  $\text{lenght}(\theta') > 1$  then
    Select a mutation type :  $t \sim \mathcal{U}([\text{add}, \text{delete}, \text{modify}])$ 
  else
    Select a mutation type :  $t \sim \mathcal{U}([\text{add}, \text{modify}])$ 
  end if
  if mutation type is add and freecycles non empty then
    newcycle  $\sim \mathcal{U}(\text{freecycles})$ 
    instruction type  $\sim \mathcal{U}([\text{read}, \text{write}])$ 
    address  $a = \mathcal{U}([a_{min}, a_{max}])$ 
     $\theta'(\text{newcycle}) = \{\text{instruction type}, a\}$ 
    remove freecycles(newcycle)
  else if mutation type is delete and  $\text{lenght}(\theta') > 1$  then
    cycle to delete  $\sim \mathcal{U}(\theta')$ 
    Delete  $\theta'(\text{cycle to delete})$ 
    set free freecycles(cycle to delete)
  else if Mutation type is modify and  $\text{lenght}(\theta') > 1$  then
    cycle to modify  $c \sim \mathcal{U}(\theta')$ 
     $t_{old}, a_{old} = \theta'(c)$ 
    modify choice  $m \sim \mathcal{U}([\text{type}, \text{adress}, \text{both}])$ 
    if  $m = \text{type}$  then
       $t_{new} = \text{write}$  if  $t_{old} == \text{read}$  else  $t_{new} = \text{read}$ 
       $\theta'(c) = \{\text{type}: t_{new}, \text{address}: a_{old}\}$ 
    else if  $m = \text{address}$  then
      new address  $a_{new} \sim \mathcal{U}(\{a_{min}, \dots, a_{max}\})$ 
       $\theta'(c) = \{\text{type}: t_{old}, \text{address}: a_{new}\}$ 
    else
      //Change both type and address
       $t_{new} = \text{write}$  if  $t_{old} == \text{read}$  else  $t_{new} = \text{read}$ 
      new address  $a_{new} \sim \mathcal{U}(\{a_{min}, \dots, a_{max}\})$ 
       $\theta'(c) = \{\text{type}: t_{new}, \text{address}: a_{new}\}$ 
    end if
  end if
  if  $\text{lenght}(\theta') > i_{max}$  then
    Uniformly sample set of distinct  $\text{lenght}(\theta') - i_{max}$  instructions to delete
    Delete the instructions
  end if
end for

```

B.4 Selection of parameters

IMGEP parameters	Description	Value
N_{init}	Number of iterations for the warming set	1000
$N - N_{init}$	Total number of IMGEP iterations	9000
k	Number of neighbors in the kNN model for the goal achievement strategy II.	1,2,3
Number of mutations	Number of mutation for each program in the model II	5
max sending cycle	Maximum cycle for sending the instruction (cycle)	60
Range instructions	Minimal and maximal number of instructions	1-10
core 0	Range of addresses	0-20
core 1	Range of addresses	21-40

Table 4: The selection of IMGEP parameters for our experiments.

Random exploration parameters	Description	Value
N	Total number of IMGEP iterations	10000
max sending cycle	Maximum cycle for sending the instruction (cycle)	60
Range instructions	Minimal and maximal number of instructions	1-10
core 0	Range of addresses	0-20
core 1	Range of addresses	21-40

Table 5: The selection of parameters for random exploration

C Evaluation

C.1 Diversity

Another diversity measure, but ill-defined, is the sum of the squared euclidean distances between all pairs: $\mathcal{D}(\mathcal{H}) = \sum_i \sum_{j < i} \|o_i - o_j\|^2$. This measure is not a well defined positive measure as $\mathcal{D}(H_1 \cup H_2) \neq \mathcal{D}(H_1) + \mathcal{D}(H_2)$, when $H_1 \cap H_2 = \emptyset$ in general. However, it provides a good intuition of the diversity, as a small perimeter cloud with small pairwise distances leads to a small diversity result whereas a large cloud perimeter involves larger pairwise distances and thus a larger diversity.